Parallel Reduction using OpenMP (Min, Max, Sum, Average)

Aim

Implement **Min, Max, Sum, and Average operations** using **Parallel Reduction in OpenMP**.

Theory

What is Parallel Reduction?

Reduction is a technique used in parallel programming to combine results from multiple threads into a single value.

Examples:

- Sum of all elements
- Minimum value
- Maximum value
- Average

Instead of one thread processing all elements sequentially, multiple threads process parts of the array simultaneously.

OpenMP provides the reduction clause to safely combine results.

OpenMP Reduction Syntax

#pragma omp parallel for reduction(operation:variable)

Example:

#pragma omp parallel for reduction(+:sum)

Each thread creates a private copy of sum.

After execution:

- OpenMP combines all thread results into final sum.

Reduction Operations Used

Operation	Symbol
Sum	+
Minimum	min
Maximum	max

Explanation of Operations

---

1. Parallel Sum

Sequential

sum += arr[i];

One thread calculates total sum.

Parallel Version

#pragma omp parallel for reduction(+:sum)

Working

- Array divided among threads
- Each thread computes partial sum
- OpenMP combines all partial sums

---

2. Parallel Minimum

#pragma omp parallel for reduction(min:minVal)

Working

Each thread:

- Finds local minimum
- OpenMP compares all local minimums
- Final minimum is stored in minVal

---

3. Parallel Maximum

#pragma omp parallel for reduction(max:maxVal)

Working

Each thread:

- Finds local maximum
- OpenMP combines results

---

4. Average

Formula:

Average = \frac{Sum}{N}

Where:

- Sum = total of all elements
- N = number of elements

Complete Program (User Input Version)

```
#include <iostream>

#include <vector>

#include <omp.h>


using namespace std;


class Reduction {


    vector<int> arr;


public:


    Reduction(vector<int> a) {

        arr = a;

    }


    void operations() {


        int n = arr.size();


        int sum = 0;


        int minVal = arr[0];


        int maxVal = arr[0];
```

```
// Parallel Sum
#pragma omp parallel for reduction(+:sum)
for(int i = 0; i < n; i++) {
    sum += arr[i];
}

// Parallel Minimum
#pragma omp parallel for reduction(min:minVal)
for(int i = 0; i < n; i++) {

    if(arr[i] < minVal) {
        minVal = arr[i];
    }
}

// Parallel Maximum
#pragma omp parallel for reduction(max:maxVal)
for(int i = 0; i < n; i++) {

    if(arr[i] > maxVal) {
        maxVal = arr[i];
    }
}

double average = (double)sum / n;

cout << "\nResults:\n";

cout << "Sum = " << sum << endl;
```

```
        cout << "Minimum = " << minVal << endl;

        cout << "Maximum = " << maxVal << endl;

        cout << "Average = " << average << endl;
    }
};

int main() {

    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> arr(n);

    cout << "Enter array elements:\n";

    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    Reduction r(arr);

    r.operations();

    return 0;
}
```

Sample Input

Enter number of elements: 5

Enter array elements:

10

20

30

40

50

Sample Output

Results:

Sum = 150

Minimum = 10

Maximum = 50

Average = 30

Time Complexity

Operation	Complexity
Sum	O(n)
Min	O(n)
Max	O(n)
Average	O(1)

Using parallelism:

- Work is divided among threads
- Execution becomes faster on multicore processors

Advantages of Parallel Reduction

- Faster execution
- Efficient CPU utilization
- Simple syntax using OpenMP

- Useful for large datasets

Important OpenMP Concepts Used

Concept	Purpose
#pragma omp parallel for	Parallelizes loop
reduction	Combines thread results safely
min reduction	Finds minimum
max reduction	Finds maximum
+ reduction	Computes sum

Compilation Command

Linux / Mac

```
g++ reduction.cpp -fopenmp
./a.out
```

Windows (MinGW)

```
g++ reduction.cpp -fopenmp -o reduction
reduction.exe
```

Viva Questions

1. What is reduction in OpenMP?

Reduction combines partial results from multiple threads into one final result.

---

2. Why reduction is needed?

To avoid race conditions while updating shared variables.

---

3. What is race condition?

When multiple threads modify the same variable simultaneously causing incorrect results.

---

4. Difference between sequential and parallel reduction?

Sequential	Parallel
Single thread	Multiple threads
Slower	Faster
No thread communication	Uses reduction mechanism

---

5. Which operations support reduction?

- Sum
- Product
- Minimum
- Maximum
- Logical operations

Conclusion

The program successfully implements:

- Parallel Sum
- Parallel Minimum
- Parallel Maximum
- Parallel Average

using OpenMP reduction clauses, improving performance using multithreading.